

知网个人查重服务报告单(全文标明引文)

报告编号:BC2026050917403316844907525

检测时间:2026-05-09 17:40:33

篇名: 基于A2A与MCP的个人健康管理系统的设计与实现

作者: 马佳辉

检测类型: 毕业论文/设计

比对截止日期: 2026-05-09

检测结果

去除本人文献复制比: 3%

去除引用文献复制比: 2.3%

总文字复制比: 3%

单篇最大文字复制比: 0.5% (2550755540976448610176_长期降水变化和氮添加对青藏高原高寒草原植_邓彤彤)

重复字符数: [900]

单篇最大重复字符数: [136]

总字符数: [29760]

0% (0)	0% (0)	中英文摘要等 (总2285字)
2.5% (71)	2.5% (71)	第1章绪论 (总2813字)
10.7% (315)	10.7% (315)	第2章相关技术与理论基础 (总2941字)
0% (0)	0% (0)	第3章系统需求分析 (总1327字)
0% (0)	0% (0)	第4章系统总体设计 (总3037字)
0.9% (39)	0.9% (39)	第5章系统详细实现 (总4448字)
4% (53)	4% (53)	第6章系统测试与结果分析 (总1335字)
3.6% (422)	3.6% (422)	第7章总结与展望 (总11574字)

(注释: 无问题部分 文字复制部分 引用部分)

1. 中英文摘要等

总字符数: 2285

相似文献列表

去除本人文献复制比: 0% (0)

去除引用文献复制比: 0% (0)

文字复制比: 0% (0)

原文内容

本科毕业论文(设计)

题目: 基于A2A与MCP的个人健康管理系统的设计与实现

姓名: 马佳辉

学号: P221513201

学院: 教育科学与技术学院

专业: 数字媒体技术

班级: 2022级数字媒体技术班

指导教师: 赵堃

2025年 5月

基于A2A与MCP的个人健康管理系统的设计与实现

专业: 数字媒体技术 姓名: 马佳辉 指导教师: 赵堃

Design and Implementation of a Personal Health Management System Based on A2A and MCP

摘要

日常生活中,很多人会有这样的困扰:体检报告、化验单、用药记录散落在不同医院和手机相册里,想看的时候找不到;去医院复诊时,医生问之前的用药情况也记不清楚。针对这些实际问题,本文开发了一套个人健康助手系统,采用A2A-MCP架构,让多个模块功能能够分工配合。

系统设计了四个智能体，分别处理健康咨询、档案整理、用药提醒和就诊摘要，智能体之间可通过A2A协议进行任务转交。工具方面，用MCP协议把OCR识别、数据库查询、医学知识检索等功能统一封装，后续加新工具按规范接入就行。系统还引入向量数据库配合检索增强生成技术，让回复能参考已有的医学资料，减少模型乱回答的情况。前端用微信小程序，后端通过SSE技术逐字输出，用户体验更好。

实际测试中，系统能正确识别药品包装盒处方图片中的文字，健康咨询能结合用户历史档案给出建议，用药提醒按设定时间推送，就诊摘要能从病例中提取关键信息。整体运行稳定，响应速度可接受。

关键词：A2A智能体架构；MCP协议；多智能体系统；RAG检索增强；个人健康档案

Abstract

In daily life, many people face similar problems: medical examination reports, lab results, and medication records are scattered across different hospitals and phone photo albums, making them hard to find when needed. During follow-up visits, patients often cannot recall their previous medication details when doctors ask. To address these practical issues, this thesis develops a personal intelligent health assistant system using the A2A-MCP architecture, enabling multiple functional modules to work together.

The system includes four agents, each handling a specific task: health consultation, record organization, medication reminders, and visit summary generation, with task transfer between agents supported through the A2A protocol. For tool integration, the MCP protocol uniformly wraps OCR recognition, database queries, and medical knowledge retrieval functions. New tools can be added by following the established specifications. The system also incorporates a vector database combined with retrieval-augmented generation technology, allowing responses to reference existing medical materials and reducing inaccurate answers. The frontend is implemented as a WeChat mini program, while the backend uses SSE technology for character-by-character output, improving user experience.

During actual testing, the system correctly recognized text from medicine packaging and prescription images. Health consultations provided suggestions based on users' historical records. Medication reminders were pushed according to the set schedule. Visit summaries extracted key information from medical records. The system ran stably with acceptable response times.

KeyWords: A2A Agent Architecture; MCP Protocol; Multi-Agent System; RAG; Personal Health Records

目录

摘要.....I

Abstract.....II

第1章绪论.....1

1.1 研究背景与意义.....1

1.2 国内外研究现状.....2

1.2.1 国外研究现状.....2

1.2.2 国内研究现状.....2

1.3 研究内容与技术路线.....3

1.4 主要贡献与创新点.....4

1.5 论文组织结构.....5

第2章相关技术与理论基础.....6

2.1 A2A多智能体协同机制.....6

2.1.1 智能体概述.....6

2.1.2 A2A协议原理.....6

2.1.3 多智能体协作模式.....7

2.2 MCP工具协议与调用机制.....7

2.2.1 MCP协议概述.....7

2.2.2 MCP协议架构.....8

2.2.3 MCP在医疗场景的应用.....8

2.3 大语言模型与流式响应技术.....9

2.3.1 大语言模型概述.....9

2.3.2 RAG检索增强生成.....9

2.3.3 流式响应技术.....10

2.4 OCR与医疗文本结构化技术.....10

2.4.1 OCR技术概述.....10

2.4.2 医疗文档OCR处理.....10

2.4.3 文本结构化提取.....10

2.5 向量数据库与语义检索技术.....11

2.5.1 向量数据库概述.....11

2.5.2 语义检索原理.....11

2.5.3 pgvector向量索引.....12

2.6 本章小结.....12

第3章系统需求分析.....13

3.1 业务场景分析	13
3.1.1 应用场景描述	13
3.1.2 用户角色分析	13
3.2 功能需求分析	13
3.2.1 健康档案管理	13
3.2.2 智能健康咨询	14
3.2.3 用药提醒管理	14
3.2.4 就诊摘要生成	14
3.3 非功能需求	14
3.3.1 安全需求	14
3.3.2 可用性需求	15
3.4 本章小结	15
第4章系统总体设计	16
4.1 系统总体架构设计	16
4.1.1 系统架构概述	16
4.1.2 技术选型说明	16
4.2 智能体模块划分与职责	17
4.2.1 健康顾问智能体 (HealthAdvisor)	17
4.2.2 健康档案管理员智能体 (HealthRecordsManager)	18
4.2.3 用药提醒助手智能体 (MedicationReminder)	18
4.2.4 就诊摘要生成器智能体 (VisitSummaryGenerator)	18
4.3 A2A任务交互设计	18
4.3.1 任务委托机制	18
4.3.2 消息格式定义	19
4.4 MCP工具层设计	20
4.4.1 MCP工具列表	20
4.4.2 工具调用流程	21
4.5 数据模型与数据库设计	22
4.5.1 数据库选型	22
4.5.2 核心数据表设计	22
4.6 安全与异常处理机制	22
4.6.1 安全机制	22
4.6.2 异常处理	23
4.7 本章小结	23
第5章系统详细实现	24
5.1 开发环境与配置	24
5.1.1 开发环境	24
5.1.2 项目结构	25
5.2 A2A服务器与任务管理实现	25
5.2.1 A2A服务器核心实现	25
5.2.2 任务管理器实现	26
5.3 MCP工具调用实现	26
5.3.1 MCP客户端实现	26
5.3.2 健康档案OCR识别实现	26
5.3.3 用药提醒工具实现	27
5.4 智能体协作与记忆系统	27
5.4.1 智能体协作实现	27
5.4.2 记忆系统实现	27
5.5 前端交互与流式响应实现	28
5.5.1 微信小程序前端实现	28
5.5.2 流式响应实现	29
5.5.3 Host Agent API实现	29
5.6 关键问题与解决方案	29
5.6.1 医疗文档结构化提取	30
5.6.2 多智能体状态同步	30
5.6.3 隐私数据保护	31
5.7 本章小结	31
第6章系统测试与结果分析	32
6.1 测试环境说明	32
6.2 部署验证测试	32
6.2.1 服务可用性测试	32

6.3 功能验证.....32

6.3.1 健康档案管理.....33

6.3.2 智能问诊.....33

6.3.3 用药提醒.....33

6.3.4 就诊摘要生成.....34

6.4 结果分析与讨论.....34

6.4.1 功能完整性分析.....34

6.4.2 局限性讨论.....34

6.5 本章小结.....34

第7章总结与展望.....35

7.1 研究工作总结.....35

7.1.1 主要完成工作.....35

7.1.2 主要创新点.....35

7.1.3 实际应用价值.....36

7.2 存在的不足.....36

7.3 未来研究展望.....36

7.4 本章小结.....36

参考文献.....38

附录.....40

A.1 A2A服务器核心代码.....40

A.2 任务管理器核心代码.....40

A.3 MCP客户端核心代码.....41

A.4 OCR识别核心代码.....42

A.5 用药提醒核心代码.....43

A.6 智能体协作核心代码.....43

A.7 记忆系统核心代码.....44

A.8 SSE流式响应核心代码.....45

A.9 Host Agent API核心代码.....46

A.10 医疗文档结构化提取核心代码.....47

A.11 多智能体状态同步核心代码.....47

A.12 隐私数据保护核心代码.....48

致谢.....50

2. 第1章绪论		总字符数：2813
相似文献列表		
去除本人文献复制比：2.5%(71) 去除引用文献复制比：2.5%(71) 文字复制比：2.5%(71)		
1	基于树莓派的二维码门禁系统的设计与实现 沈丞锋 - 《大学生论文联合比对库》- 2018-04-03	1.4% (40) 是否引证：否
2	酒店管理系统的设计与实现 廖丹 - 《大学生论文联合比对库》- 2021-06-01	1.4% (39) 是否引证：否
3	2021302111134-应志伟-自动驾驶周视3D目标检测.pdf 应志伟 - 《大学生论文联合比对库》- 2025-05-19	1.1% (31) 是否引证：否
4	2021302111134-应志伟-自动驾驶周视3D目标检测 应志伟 - 《大学生论文联合比对库》- 2025-04-24	1.1% (31) 是否引证：否

原文内容

第1章绪论

1.1 研究背景与意义

现在越来越多人开始关注自身健康、慢病管理、日常健康咨询、用药提醒、就诊后信息整理这些需求也跟着多了起来[1]。以前大家管理健康主要靠跑医院和手工记录，但体检报告、化验单、用药记录往往散落在不同地方，想看的时候找不到，出了问题也没法及时反馈，长期跟踪更困难。这几年大语言模型在理解自然语言、知识问答和多轮对话方面有很明显的进步，让“个人智能健康助手”从概念变成了可以实际落地的产品[2]。

不过在实际使用的过程中，光靠一个对话模型还不够。健康问题往往涉及多个方面，比如查档案、看报告、评估风险、设提醒等，这些任务的性质不一样，一个模型很难全部处理好。另外系统还要对接OCR识别，数据库查询、知识检索、任务调度等外部功能，如果所有功能都塞在一个模块里面，代码耦合度高，后续加新功能或者改旧逻辑都很麻烦[3]。

针对这些问题，多智能体协同和工具化调用成了比较可行的解决办法。A2A机制让智能体之间通过统一协作，适合把复杂健康服务拆成多个职责明确的Agent，MCP机制则是把工具能力标准化，给模型调用外部工具统一接口[4]。两者结合，一个解决“任务怎么分”，一个解决“能力怎么接”，正好能支撑智能健康助手系统的构建。

本课题面向个人健康管理场景，做了一套基于A2A-MCP的个人智能健康助手系统。系统围绕着“健康咨询、健康档案管理、用药提醒、就诊摘要生成”四个核心功能搭建，通过多智能体架构在健康场景中的落地提供了一个可供参考的方案。

本研究的意义主要体现在三个方面。在理论与方法方面，把A2A协同和MCP工具协议结合起来，形成了一套面向复杂健康任务的系统化设计方法。在工程实现方面，通过模块化架构把各功能解耦，如果系统后面增加新功能或者改旧逻辑会变得方便。在应用实践方面，围绕用户实际使用链路提供完整的功能闭环，让健康管理更便捷、更连续。

1.2 国内外研究现状

1.2.1 国外研究现状

国外做智能健康助手的时间比较早，整体经历了从“规则问答”到“对话系统”再到“大模型代理系统”几个阶段[5]。最早的系统主要靠固定知识库和模板回复，在特定问题上回答比较可控，但凡换个问法或者遇到没覆盖到的场景就不行了。后来深度学习和预训练模型出来以后，系统在语义理解和自然语言生成方面有了明显的提升，对话也更自然了。

近几年国外在多智能体和工具调用方面的研究多了起来。一部分工作关注怎么把任务拆给不同代理，让它们并行或串行执行；另一部分关注函数调用和外部工具接入，通过统一接口让模型能查数据库、跑代码、解析文档等[6]。这些方法在复杂场景下确实提高了任务完成率，而且执行过程更加可以解释。

1.2.2 国内研究现状

国内这方面的研究和应用发展也挺快，特别是在医疗健康、政务服务、企业客服这些领域，结合大模型的垂直系统越来越多[7]。健康方向常见的功能包括医疗知识问答、体检报告解读、慢病随访提醒、电子病历信息抽取等。总的来看，国内在场景落地和工程部署方面推进比较快，但在多智能体标准协同协议、工具接口统一规范、系统可扩展性验证这些方面还有提升空间[8]。

综合现有研究，目前还存在几个共性问题。单Agent架构在功能变多后职责边界不清，系统复杂度跟着快速上升。工具接入方式没有统一标准，跨模块调用链路长、复用性不够。对系统性能、稳定性和可扩展性的评估不够充分，很多论文更注重算法效果，工程验证相对较少。

因此，本课题以A2A-MCP为核心，强调“协议统一加模块解耦加工程验证”，在个人智能健康助手场景中提供一套从架构设计到系统实现再到测试评估的完整方案。

1.3 研究内容与技术路线

本课题的目标是做一套能实际使用的个人智能健康助手系统，满足健康咨询和信息管理场景下的多任务协同需求。围绕这个目标，主要做了四个方面的工作。搭建多智能体协同架构，按业务职责划分了健康顾问、健康档案管理员、用药提醒助手、就诊摘要生成器四个Agent，通过A2A协议让它们能互相传消息和更新状态。搭建统一工具调用的方法，使用MCP协议把OCR、操作数据库、知识检索等工具封装起来，使得Agent在调用外部功能的时候有统一的标准。设计端到端的任务链路，从用户发请求、任务解析、流式响应、工具回调到结果汇总，整个流程都跑通了，消息和附件也可以持久化保存了。做系统测试与评估，围绕功能正确性做了测试，验证方案是否可行。

在技术实现上，系统使用前后端分离和微服务化部署；后端用Python和异步框架完成A2A服务和任务管理，前端有消息交互和附件上传的能力，数据库负责存储数据，SSE用于流式响应。系统通过Docker部署，方便统一管理和复现实验环境[9]。

本文技术路线可以概括为：需求分析→架构设计→协议与模块实现→联调测试→结果评估与总结。具体流程如图 1-1 论文研究技术路线图具体过程包括需求阶段明确用户场景、功能边界和非功能指标，设计阶段完成总体架构、模块职责和数据模型设计，实现阶段完成了A2A任务请求的分发、任务管理、MCP工具调用、流式响应与持久化存储，测试阶段完成功能用例测试并进行结果分析。

图 1-1 论文研究技术路线图

1.4 主要贡献与创新点

相比只有单一问答能力的智能系统，本文着重“系统级构建与落地”，主要贡献包括四个方面。做了面向个人健康场景的A2A-MCP融合架构，通过智能体分工和协议化工具的接入，把复杂健康任务拆成了可协同、可扩展的执行单元。设计了流式任务处理和状态反馈机制，系统支持任务提交、处理中间状态回传、工具调用结果回传与最终结果汇总，交互更加的实时，链路也更透明。实现了健康业务中关键能力闭环，围绕着健康咨询、档案管理、OCR识别、用药提醒、就诊摘要等功能完成端到端打通，功能比较完整[10]。从工程角度验证了方案的可行性，通过功能论证该方案在真实场景中的可部署价值。

本文的创新点主要体现在以下两个层面：一是架构创新，即以A2A与MCP双协议协同解决多任务健康服务的系统复杂性；二是工程创新，即在统一任务链路中融合流式响应、工具回环和持久化机制，形成可复用的实践案例。

1.5 论文组织结构

全文结构安排如下。第1章为绪论，介绍研究背景、现状、研究内容、贡献与论文组织。第2章介绍涉及的关键技术与理论基础，包括A2A、MCP、大模型、OCR等。第3章进行系统需求分析，给出功能需求、非功能需求及业务流程。第4章阐述系统总体设计，包括架构设计、模块划分、交互协议与数据设计。第5章给出系统详细实现，重点说明任务流、工具调用、附件处理与持久化实现。第6章展示测试方案与实验结果，并对功能可用性分析。第7章总结全文工作，指出不足并展望后续研究方向。

通过上述章节安排，本文形成了从问题提出、方案设计、系统实现到实验验证的完整研究闭环。

3. 第2章相关技术与理论基础			总字符数：2941
相似文献列表			
去除本人文献复制比：10.7%(315)		去除引用文献复制比：3.6%(107)	文字复制比：10.7%(315)

1	中国矿业大学毕业设计（论文）_dyk_定稿.pdf 邓岳康 - 《大学生论文联合比对库》 - 2025-06-02	3.7% (110) 是否引证：否
2	字符识别应用场景. - 《网络（ https://www.360docs. ）》 - 2025	2.7% (79) 是否引证：否
3	基于视觉的升压站表计读数识别算法设计 王曦 - 《大学生论文联合比对库》 - 2025-05-19	1.8% (52) 是否引证：否
4	基于视觉的升压站表计读数识别算法设计 王曦 - 《大学生论文联合比对库》 - 2025-06-11	1.8% (52) 是否引证：否
5	结合GLM和医疗领域知识图谱问答系统的设计与实现 刘少杰 - 《大学生论文联合比对库》 - 2024-05-24	1.8% (52) 是否引证：否
6	结合GLM和医疗知识图谱的智能问答系统设计与实现 刘少杰 - 《大学生论文联合比对库》 - 2024-05-31	1.8% (52) 是否引证：否
7	面向AIGC应用的安全性检测技术的研究与实现 薛子骆 - 《大学生论文联合比对库》 - 2024-05-20	1.8% (52) 是否引证：否
8	4acdb7f995884633a916c90e0fa0b180.docx 顾贤 - 《大学生论文联合比对库》 - 2024-05-31	1.4% (41) 是否引证：否
9	基于大模型的个人财务管理系统的设计与实现 张一鸣 - 《大学生论文联合比对库》 - 2025-05-25	1.4% (41) 是否引证：否
10	3210103796_储宁奕_基于神经网络的汉字笔画auto-tracing 储宁奕 - 《大学生论文联合比对库》 - 2025-05-15	1.0% (28) 是否引证：否
11	基于大模型的漏洞重复性提示与危害性评估研究 李庆航 - 《大学生论文联合比对库》 - 2024-06-16	1.0% (28) 是否引证：否

原文内容

第2章相关技术与理论基础

2.1 A2A多智能体协同机制

2.1.1 智能体概述

智能体（Agent）是一个可以理解成能自己感知环境、做出决策并执行动作的独立实体。在人工智能领域，基于大语言模型的智能体（LLM Agent）能看懂自然语言输入、做逻辑推理、规划任务步骤，还能调用外部工具完成复杂目标[3]。一个智能体通常包含感知模块、推理引擎、工具调用接口和记忆组件构成，形成“感知-规划-行动”的闭环。

2.1.2 A2A协议原理

A2A（Agent-to-Agent）协议是用来解决智能体之间怎么通信的标准，主要处理多智能体系统中的任务分发、状态同步和协作的问题[4]。在A2A架构里，每个智能体既可以是发起任务的一方，也可以是执行的一方，大家通过统一的消息格式来交互。

A2A协议主要有任务委托机制、状态同步和结果汇聚三个特性。任务委托机制指智能体把复杂任务拆开，委托给其他专业智能体处理，各干各的。状态同步通过消息队列或事件总线让智能体之间的状态保持一致。结果汇聚则是多个智能体的处理结果由协调者收集总结后返回给用户。

A2A协议的交互流程如图 2- 1 A2A协议交互流程图。

图 2- 1 A2A协议交互流程图

2.1.3 多智能体协作模式

多智能体协作有三种方式：串行、并行和混行。串行适合任务之间有依赖的情况，一个做完下一个才能开始；并行适合子任务互相独立的情况，可以同时做；混合则结合两者，根据任务特点动态选择。

在个人健康助手场景里，用户发一个健康咨询请求，可能要查历史档案、分析症状、检查用药等多个子任务，比较适合用混用协作模式。

2.2 MCP工具协议与调用机制

2.2.1 MCP协议概述

MCP（Model Context Protocol）模型上下文协议，是一种用来连接大语言模型和外部工具、数据的标准化协议[6]。MCP定义了模型和外部能力交互的标准接口，让AI系统能够灵活调用各类工具，不用给每个工具编写单独写代码。

2.2.2 MCP协议架构

MCP协议用的是客户端-服务器架构，主要包含MCP Host（AI应用的运行环境，也是用户接触的部分）、MCP Client（跑在Host端，负责跟MCP Server建立连接和维护会话）和MCP Server（封装特定工具能力的独立服务进程，提供标准化的工具调用接口）三个部分[6]。

MCP协议用JSON Schma来描述工具，让AI模型能够准确理解工具用途、参数要求和返回值格式。这种标准化设计大大降低了工具接入成本，后续新加工具就像插U盘一样方便。MCP协议的架构如图 2- 2 MCP协议架构图所示。

图 2- 2 MCP协议架构图

2.2.3 MCP在医疗场景的应用

在医疗健康场景里，MCP协议可用来标准化接入OCR识别服务、医疗知识库、药品数据库、检查结果解读等外部能力[8]。本系统基于MCP协议封装了OCRTool、ReminderTool、DocumentTool等专业工具，实现了工具能力的即插即用。

2.3 大语言模型与流式响应技术

2.3.1 大语言模型概述

大语言模型（Large Language Model, LLM）是一类基于深度学习的大型神经网络模型，通过在海量文本数据上预训练去学习语言的通用表示，能够完成文本生成、问答、翻译、摘要等多种自然语言处理任务[9]。目前常见的大语言模型包括GPT系列、Claude系列、DeepSeek系列、Qwen系列等。

2.3.2 RAG检索增强生成

检索增强生成（RAG）技术通过从外部知识库检索相关信息来提升大语言模型的回答质量，有效缓解了大语言模型知识过时、幻觉回答等问题[10][11]。RAG技术有效缓解了大语言模型知识过时、幻觉回答等问题，使模型能够生成更准确、更加实时的内容。

RAG的核心流程包括查询编码、向量检索、上下文组装和答案生成。RAG技术的核心流程如图 2- 3 RAG技术流程图所示。本系统用向量数据库存储健康知识库，通过语义检索拿到相关文档片段，再结合用户问题生成回答。

图 2- 3 RAG技术流程图

2.3.3 流式响应技术

流式响应（Streaming Response）是指服务器以数据流的形式持续往客户端发送响应内容，而不是等全部结果生成后再一次性返回。这项技术对于需要长时间处理的任务，这项技术能明显提升用户体验。

本系统采用Server-Sent Events（SSE）技术实现流式响应。当用户发起健康咨询请求后，系统实时推送处理进度和中间结果，用户不用等完整回答即可看到逐步生成的内容。

2.4 OCR与医疗文本结构化技术

2.4.1 OCR技术概述

OCR（Optical Character Recognition）光学字符识别技术，是指用电子设备对印刷或手写文本图像进行分析处理，识别并转换成机器可编辑的数字文本的技术[12]。现在的OCR技术结合深度学习算法，在印刷体识别方面准确率已经比较高了。

2.4.2 医疗文档OCR处理

医疗文档具有格式多样、专业术语多、手写体难识别等特点，对OCR技术要求更高[13]。医疗文档的OCR处理流程如图 2- 4 OCR处理流程图所示。本系统采用阿里云OCR服务，支持病历、处方、检查报告等多种医疗文档的文字识别。

2.4.3 文本结构化提取

在OCR识别完之后，还需要对文本进行结构化处理，提取关键信息比如患者信息、诊断名称、检查指标、药物名称等[14]。本系统结合规则引擎与AI大模型进行实体识别和关系抽取，把非结构化的医疗文档转化成结构化数据进行存储。

图 2- 4 OCR处理流程图

2.5 向量数据库与语义检索技术

2.5.1 向量数据库概述

向量数据库是专门用来存储和检索高维向量表示的数据库系统，支持基于语义相似度的近似最近邻搜索[15]。跟传统关系数据库不一样，向量数据库通过计算向量间的距离（比如余弦相似度、欧氏距离）来衡量语义相关性。

2.5.2 语义检索原理

语义检索利用深度学习模型把文本转换为高维向量，通过向量相似度计算实现语义级别的匹配[16]。向量检索的原理如图 2- 5 向量检索原理图所示。这种方法能够发现表述不同但语义相似的内容，克服了关键词匹配只看字面的局限性。

2.5.3 pgvector向量索引

pgvector是PostgreSQL数据库的向量索引扩展，支持在关系数据库中存储向量并进行相似度检索[15]。本系统使用pgvector实现健康知识的向量化存储和语义检索，兼顾SQL查询的灵活性和向量检索的准确性。

图 2- 5 向量检索原理图

2.6 本章小结

本章介绍了本文涉及的关键技术与理论基础。A2A协议给多智能体协作提供了标准化的通信机制，让复杂健康任务能分解和协同；MCP协议标准化了工具调用接口，实现能力的即插即用；大语言模型与RAG技术提供了智能问答和知识检索能力；OCR技术实现了医疗文档的数字化；向量数据库支撑了语义检索功能。这些技术为本系统的设计与实现打下了坚实基础。

4. 第3章系统需求分析

总字符数：1327

相似文献列表

去除本人文献复制比：0%(0)

去除引用文献复制比：0%(0)

文字复制比：0%(0)

原文内容

第3章系统需求分析

3.1 业务场景分析

3.1.1 应用场景描述

数字健康技术在主动健康领域的应用正在快速发展[1]。个人智能健康助手主要面向以下几个典型应用场景。日常健康咨询

场景中，用户感觉不舒服时可以直接用自然语言描述症状，系统会结合用户的健康档案给出初步分析和建议。健康档案管理场景中，用户拍个照或者上传病历、检查报告、处方等医疗文档，系统自动识别文字并结构化存起来。用药提醒场景中，根据处方信息设置服药提醒，记录每次服药情况。就诊前准备场景中，去看病之前可以用系统生成就诊摘要，帮用户整理好病史、主诉等信息，跟医生沟通更高效。

3.1.2 用户角色分析

系统涉及的主要用户角色包括普通用户（用健康助手管理自己的健康档案，做健康咨询和用药管理）、健康顾问智能体（负责症状分析和健康建议）、档案管理员智能体（处理健康档案的录入、OCR识别和结构化提取）、用药提醒智能体（管理用药计划，发提醒通知）和摘要生成智能体（整合健康信息，生成就诊前摘要）。

3.2 功能需求分析

3.2.1 健康档案管理

健康档案管理功能包括多模态录入（支持拍照上传、现有图片上传以及手动输入三种方式）、OCR识别（调用阿里云OCR服务自动识别医疗文档中的文字）、结构化提取（从OCR识别出来的文本中提取关键字段比如患者姓名、诊断、药物、检查指标等）、安全存储（敏感健康数据加密存储保护用户隐私）和档案查询（可以按类型进行条件筛选和按照关键字搜索档案）[18]。

3.2.2 智能健康咨询

基于知识图谱的医疗诊断智能问答系统能够整合规范化医学知识，为用户提供可溯源的医疗诊断建议[17]。智能健康咨询功能包括症状描述（用户用自然语言描述身体不适的症状）、病史参考（系统结合用户过往健康档案给出个性化建议）、健康知识问答（回答关于常见疾病、药物作用、健康生活方式等问题）和就医建议（在分析基础上给出就医建议，同时强调仅供参考，不能替代专业诊断）。

3.2.3 用药提醒管理

用药提醒管理功能包括提醒设置（根据用药频率和时间自动创建提醒计划）、服药记录（用户确认服药后系统记录服药时间和状态）。

3.2.4 就诊摘要生成

就诊摘要生成功能包括信息整合（汇总用户近期的健康咨询、档案和用药情况）、摘要生成（生成就诊前摘要包含主诉、病史、用药史、近期检查等）。

3.3非功能需求

3.3.1 安全需求

医学人工智能研究在伦理审查方面面临诸多挑战，包括算法偏见、数据隐私、责任归属等问题需要重点关注[7]。医疗数据涉及用户隐私，系统需要满足数据加密（敏感健康数据用AES-256加密存储）、传输安全（全站用HTTPS加密传输，计划中需要申请SSL证书）、访问控制（基于JWT做用户身份认证和权限管理）和数据最小化（只采集和存储必要的健康数据）等安全要求[7]。

3.3.2 可用性需求

可用性需求包括多支持微信小程序访问、通过SSE实现流式响应，支持消息推送通知和有异常处理和错误提示能力，保证系统稳定运行。

3.4 本章小结

本章分析了系统的业务场景和功能需求，明确了健康档案管理、智能健康咨询、用药提醒、就诊摘要生成四大核心功能模块。

5. 第4章系统总体设计

总字符数：3037

相似文献列表

去除本人文献复制比：0%(0)

去除引用文献复制比：0%(0)

文字复制比：0%(0)

原文内容

第4章系统总体设计

4.1 系统总体架构设计

4.1.1 系统架构概述

本系统采用A2A-MCP双核架构，整个系统分成四个层次：接入层、A2A智能体层、MCP工具层和数据层[4]。其中接入层提供微信小程序前端和Web管理后台，通过FastAPI网关接入系统，支持SSE流式响应。A2A智能体层包含了四个核心的智能体——健康顾问、健康档案管理员、用药提醒助手、就诊摘要生成器，通过A2A协议进行任务协作。MCP工具层封装各类外部能力的标准化接口，包括OCRTool、ReminderTool、DocumentTool、KnowledgeTool等。数据层包括PostgreSQL关系数据库存储业务数据，pgvector向量数据库存储语义记忆，阿里云OCR提供文字识别能力[4]。

图 4- 1 系统架构图

系统架构图如图 4- 1 系统架构图所示。

4.1.2 技术选型说明

后端用Python开发，使用Python丰富的AI生态和异步编程；A2A服务基于A2AServer框架实现（基于JSON-RPC2协议）；MCP工具基于FastMCP框架封装；数据库选用PostgreSQL结合pgvector扩展；前端使用微信小程序开发。

系统技术选型如表 4- 1 技术选型表所示。

表 4- 1 技术选型表

层次	技术选型	说明
----	------	----

接入层	微信小程序+FastAPI	原生开发，支持SSE流式响应
接入层	Google ADK	HostAgent开发框架
A2A层	A2AServer(自实现)	基于JSON-RPC2的A2A协议服务器
MCP层	FastMCP	专业智能体工具封装框架
MCP层	LiteLLM	统一LLM调用接口
数据库	PostgreSQL + pgvector	业务数据+向量检索
缓存	Redis	消息队列、分布式锁
后端语言	Python 3.12	异步编程+AI生态
部署	Docker Compose	容器化部署

层次技术选型说明

接入层微信小程序+FastAPI 原生开发，支持SSE流式响应
接入层 Google ADK HostAgent开发框架
A2A层 A2AServer(自实现) 基于JSON-RPC2的A2A协议服务器
MCP层 FastMCP 专业智能体工具封装框架
MCP层 LiteLLM 统一LLM调用接口
数据库 PostgreSQL + pgvector 业务数据+向量检索
缓存 Redis 消息队列、分布式锁
后端语言 Python 3.12 异步编程+AI生态
部署 Docker Compose 容器化部署

4.2 智能体模块划分与职责

4.2.1 健康顾问智能体 (HealthAdvisor)

健康顾问是系统中的专业对话智能体，由Host Agent根据用户请求类型进行调度，负责处理日常健康咨询任务[3][4]。被Host Agent调度后，健康顾问接收用户的健康咨询请求，调用MCP工具获取专业能力（如健康知识检索、药品查询等），然后向用户返回健康建议和回答。

健康顾问智能体架构如图 4- 2 健康顾问智能体架构图所示。

图 4- 2 健康顾问智能体架构图

4.2.2 健康档案管理员智能体 (HealthRecordsManager)

健康档案管理员主要负责处理用户上传的医疗文档，进行OCR识别，从识别出来的文本中提取结构化信息，并管理用户健康档案的增删改查[13]。

4.2.3 用药提醒助手智能体 (MedicationReminder)

用药提醒助手主要负责管理用户用药计划，包括添加、修改、删除药品，根据用药计划创建定时提醒[18]。

4.2.4 就诊摘要生成器智能体 (VisitSummaryGenerator)

就诊摘要生成器主要负责整合用户指定时间范围内的健康咨询记录，提取过往病史、过敏史、用药史等关键信息，生成就诊前摘要[16]：

4.3 A2A任务交互设计

4.3.1 任务委托机制

融合大语言模型与模型上下文协议技术，实现了多智能体间的任务协作[4]当用户请求涉及多个专业领域时，Host Agent作为协调者拆开并委托给相应的远程智能体处理。A2A任务交互时序图如图 4- 3 A2A任务交互时序图所示。任务委托流程为：用户发送请求到Host Agent后，Host Agent分析请求并识别需要调用的远程智能体，然后通过A2A协议向目标智能体发送任务请求，目标智能体（如健康顾问、档案管理员等）执行任务并返回结果，最后Host Agent汇总结果后返回给用户。

图 4- 3 A2A任务交互时序图

4.3.2 消息格式定义

A2A消息采用JSON格式，包含任务描述、上下文信息、用户画像等字段[4]。消息格式定义如图 4- 4 消息格式所示。

图 4- 4 消息格式

4.4 MCP工具层设计

4.4.1 MCP工具列表

表 4- 2 MCP工具列表

工具名称	功能描述	所属智能体
OCRTool	阿里云OCR文字识别	健康档案管理员
DataExtractionTool	医疗文档结构化提取	健康档案管理员
StorageTool	敏感健康数据加密存储	健康档案管理员
KnowledgeTool	医疗知识库查询	健康顾问
DatabaseTool	数据库操作	健康顾问/就诊摘要生成器
MemoryIntegrationTool	记忆系统集成	各智能体
ReminderTool	用药提醒计划管理	用药提醒助手
NotificationTool	消息推送通知	用药提醒助手
DrugSafetyTool	药品安全检查	用药提醒助手
DocumentTool	医疗文档解析	就诊摘要生成器
AIAnalysisTool	AI辅助分析	就诊摘要生成器
A2AIntegrationTool	A2A协议集成	健康顾问
AsyncAnalysisTool	异步任务分析	健康档案管理员

- 工具名称功能描述所属智能体
- OCRTool 阿里云OCR文字识别健康档案管理员
- DataExtractionTool 医疗文档结构化提取健康档案管理员
- StorageTool 敏感健康数据加密存储健康档案管理员
- KnowledgeTool 医疗知识库查询健康顾问
- DatabaseTool 数据库操作健康顾问/就诊摘要生成器
- MemoryIntegrationTool 记忆系统集成各智能体
- ReminderTool 用药提醒计划管理用药提醒助手
- NotificationTool 消息推送通知用药提醒助手
- DrugSafetyTool 药品安全检查用药提醒助手
- DocumentTool 医疗文档解析就诊摘要生成器
- AIAnalysisTool AI辅助分析就诊摘要生成器
- A2AIntegrationTool A2A协议集成健康顾问
- AsyncAnalysisTool 异步任务分析健康档案管理员

系统封装的MCP工具如表 4- 2 MCP工具列表所示。

4. 4. 2 工具调用流程

图 4- 5 MCP工具调用流程图

MCP工具调用流程如图 4- 4 MCP工具调用流程图所示。智能体根据任务需要调用MCP工具后，MCP Client把请求转发给对应的MCP Server，MCP Server执行工具逻辑并返回结果，最后MCP Client把结果返回给智能体。

4. 5 数据模型与数据库设计

4. 5. 1 数据库选型

系统用PostgreSQL作为主数据库，利用pgvector扩展支持向量检索，用来存储语义记忆和实现RAG功能[15]。

4. 5. 2 核心数据表设计

核心数据表包括：

- (1) users：用户基本信息表
- (2) user_profiles：用户健康档案表（包含过敏史、慢病、手术史等）
- (3) health_records：健康档案主表
- (4) user_medications：用户用药记录表
- (5) medication_reminders：用药提醒计划表
- (6) conversations：会话记录表
- (7) visit_summaries：就诊摘要表

4. 6 安全与异常处理机制

4. 6. 1 安全机制

系统采用JWT做用户身份认证，Token有效期设置为24小时，敏感健康数据用AES-256加密存储，并记录所有敏感操作以便安全审计。

4. 6. 2 异常处理

异常处理机制包括智能体超时处理（智能体任务设置超时机制，超时后返回友好提示）、服务降级（当某个MCP工具不可用时，系统记录日志并返回降级响应）和重试机制（对关键操作实现指数退避重试策略）。

4. 7 本章小结

本章阐述了系统总体设计方案。系统采用A2A-MCP双核架构，将智能体协作与工具调用分离，实现了职责清晰、易于扩展的系统结构。通过对四大智能体的功能划分、MCP工具的设计、数据模型的设计，为后续详细实现奠定了基础。

6. 第5章系统详细实现			总字符数：4448
相似文献列表			
去除本人文献复制比：0.9%(39) 去除引用文献复制比：0.9%(39) 文字复制比：0.9%(39)			
1	陈敬文_2020010746_bo1t. SE：基于大语言模型的软件开发辅助工具 2020010746 - 《大学生论文联合比对库》 - 2025-05-22		0.9%（39） 是否引证：否
原文内容			

第5章系统详细实现

5. 1 开发环境与配置

随着软件工程的快速发展，选择合适的开发环境和工具对于提高开发效率具有重要意义。本系统采用前后端分离的架构模式，后端使用Python语言结合FastAPI框架进行开发，前端采用微信小程序技术栈。

5. 1. 1 开发环境

表 5- 1 开发环境配置表

类别	工具/版本	说明
----	-------	----

编程语言	Python 3.12.7	后端主要开发语言
编程语言	JavaScript	微信小程序前端
数据库	PostgreSQL 16+	业务数据存储
缓存/消息队列	Redis 7+	会话状态、消息队列
容器化	Docker	服务部署
容器编排	Docker Compose	多服务编排
后端框架	FastAPI	API网关
后端框架	FastMCP	MCP工具服务器
LLM调用	LiteLLM	统一-LLM接口
前端框架	微信小程序	移动端应用
前端框架	React 18+	管理后台（Docker构建）

类别工具/版本说明

编程语言 Python 3.12.7 后端主要开发语言
编程语言 JavaScript 微信小程序前端
数据库 PostgreSQL 16+ 业务数据存储
缓存/消息队列 Redis 7+ 会话状态、消息队列
容器化 Docker 服务部署
容器编排 Docker Compose 多服务编排
后端框架 FastAPI API网关
后端框架 FastMCP MCP工具服务器
LLM调用 LiteLLM 统一-LLM接口
前端框架微信小程序移动端应用
前端框架 React 18+ 管理后台（Docker构建）

系统开发环境包括Python 3.10+、PostgreSQL 16+、Redis 7+、Docker等。详细环境配置可参考项目根目录下的docker-compose.yml和backend目录下的配置文件。

开发环境配置如表 5- 1 开发环境配置表所示。

5.1.2 项目结构

项目目录结构如图 5- 1 代码目录结构所示。

图 5- 1 代码目录结构

5.2 A2A服务器与任务管理实现

智能体服务器是整个系统的核心组件，负责处理用户请求、管理任务状态和协调工具调用。在多智能体系统中，任务管理器的设计直接影响系统的并发处理能力和响应速度。

5.2.1 A2A服务器核心实现

A2A服务器使用FastAPI框架实现，负责接收用户请求、管理智能体注册、处理任务分发和结果汇聚。A2A服务器核心代码见附录A.1。

核心代码结构主要包括BasicAgent类，包含servers（MCP服务器实例）、all_functions（可用工具列表）和session_conversations（会话历史）等属性，以及setup_tools（初始化MCP工具连接）和handle_task（处理A2A任务）等方法。

5.2.2 任务管理器实现

任务管理器负责跟踪任务状态、管理任务队列、支持任务取消和结果查询。核心实现位于‘backend/A2AServer/src/A2AServer/task_manager.py’，继承自‘InMemoryTaskManager’类。任务管理器核心代码见附录A.2。AgentTaskManager类实现了on_send_task方法，处理同步任务请求，包括请求验证、任务存储、状态更新和结果返回等步骤。

5.3 MCP工具调用实现

5.3.1 MCP客户端实现

模型上下文协议（Model Context Protocol）作为一种标准化的工具接入协议，为大语言模型提供了统一的工具调用接口。通过MCP协议，系统可以将各种外部能力封装为标准化工具，供智能体按需调用。

MCP客户端位于‘backend/A2AServer/src/A2AServer/mcp_client/client.py’，负责与MCP服务器建立连接、调用工具并处理响应。支持多种LLM提供商（DeepSeek、OpenAI、Anthropic等）。

MCP客户端核心代码见附录A.3。SSEMCPCClient类实现了start方法建立SSE连接，以及list_tools方法列出可用工具。

5.3.2 健康档案OCR识别实现

健康档案管理员通过OCRTool调用阿里云OCR服务进行文字识别。核心实现位于‘backend/HealthRecordsManager/mcpserver/ocr_tool.py’。

OCR识别核心代码见附录A.4。call_aliyun_ocr函数接收base64编码的图片，调用阿里云OCR API进行识别，返回识别文本。

核心流程：（1）用户上传医疗文档图片；（2）系统把图片base64编码后调用OCRTool；（3）阿里云OCR返回识别文本；（4）DataExtractionTool从文本中提取结构化信息；（5）数据加密后存到数据库。

5.3.3 用药提醒工具实现

用药提醒助手通过ReminderTool管理用药计划和发送提醒。核心实现位于‘backend/MedicationReminder/mcpserver/reminder_tool.py’。

用药提醒核心代码见附录A.5。create_medication_reminder函数通过PostgreSQL连接将提醒记录插入到reminders表中。

5.4 智能体协作与记忆系统

5.4.1 智能体协作实现

智能体之间通过A2A协议进行任务委托和结果交换。核心实现位于

‘backend/HealthAdvisor/mcpserver/a2a_integration_tool.py’。

智能体协作核心代码见附录A.6。_send_task函数构造JSON-RPC 2.0格式的请求体，通过HTTP POST发送到目标智能体的/unary端点。a2a_send_task是封装的MCP工具，供健康顾问调用其他智能体。

5.4.2 记忆系统实现

系统集成AgentMemorySystem实现语义记忆存储和检索。核心实现位于

‘backend/AgentMemorySystem/memory_manager.py’ 和相关模块。

记忆系统核心代码见附录A.7。MemoryManager类实现了add_memory（添加新记忆）和retrieve_similar_memories（检索相似记忆）两个核心方法。核心组件包括embedding_service.py（文本向量化服务，调用LLM API把用户对话和知识库文本转成向量）、memory_storage.py（向量存储管理，调用pgvector存储和索引向量）和memory_retrieval.py（语义检索，根据查询向量检索相似记忆）。

5.5 前端交互与流式响应实现

5.5.1 微信小程序前端实现

微信小程序作为一种轻量级应用形态，具有无需安装、即用即走的特点，近年来在医疗健康领域得到了广泛应用。本系统前端采用微信小程序技术栈实现，为用户提供便捷的健康管理服务入口。

使用微信小程序制作用户界面，支持消息交互、附件上传、消息接收等功能。微信小程序主要界面如图 5- 2 微信小程序界面截图所示。

图 5- 2 微信小程序界面截图

主要页面：

- (1) 首页：健康助手入口，支持快捷功能和消息列表；
- (2) 健康咨询页：与智能体进行对话交互；
- (3) 档案管理页：查看和管理个人健康档案；
- (4) 用药提醒页：管理用药计划和查看提醒；
- (5) 就诊摘要页：查看就诊摘要；

5.5.2 流式响应实现

系统采用Server-Sent Events (SSE) 实现服务端流式推送。核心实现位于

‘backend/A2AServer/src/A2AServer/agent.py’。

SSE流式响应核心代码见附录A.8。_stream_response_generator方法通过异步生成器逐块返回响应，支持普通文本、推理内容和工具调用结果三种类型。

5.5.3 Host Agent API实现

Host Agent API位于 ‘frontend/hostAgentAPI/api.py’，提供智能体管理的RESTful接口。

Host Agent API核心代码见附录A.9。应用启动时执行_warmup_optional_tools进行预热，ConversationServer类提供创建会话等接口。

主要接口：

接口	方法	功能
/pin	GET	健康检查
/conversation/create	POST	创建会话
/message/send	POST	发送消息
/conversation/list	POST	获取会话列表

接口方法功能

- /pin GET 健康检查
- /conversation/create POST 创建会话
- /message/send POST 发送消息
- /conversation/list POST 获取会话列表

5.6 关键问题与解决方案

5.6.1 医疗文档结构化提取

问题：医疗文档格式多样，传统规则难以应对复杂场景。

解决方案：结合规则引擎与AI大模型，先用规则提取明确字段，再调用大模型处理非结构化内容。核心实现位于

‘backend/HealthRecordsManager/mcpserver/data_extraction_tool.py’。

医疗文档格式多样，传统规则很难面对复杂场景。系统结合规则引擎与AI大模型，先用规则提取明确字段，再调用大模型处理非结构化内容。核心实现位于 ‘backend/HealthRecordsManager/mcpserver/data_extraction_tool.py’。医疗文档结构化提取核心代码见附录A.10。extract_medical_info函数从OCR文本中提取患者信息、诊断、药物、检查结果等结构化字段，detect_document_type函数通过关键词匹配检测文档类型。

5.6.2 多智能体状态同步

问题：多智能体并行工作时需要保证状态一致性。

解决方案：采用消息队列机制实现最终一致性，智能体状态变更通过事件驱动同步。

多智能体并行工作时需要保证状态一致性。系统通过A2A协议的事件驱动机制实现状态同步，每个智能体维护独立状态，通

过消息传递保持一致。多智能体状态同步核心代码见附录A. 11。AgentTaskManager通过upsert_task和update_store方法管理任务状态，从WORKING到COMPLETED的状态转换。

5.6.3 隐私数据保护

问题：健康数据涉及用户隐私，需要严格保护。

解决方案：通过A2A协议的事件驱动机制实现状态同步，每个智能体维护独立状态，通过消息传递保持一致。

健康数据涉及用户隐私，需要严格保护。系统采用Fernet对称加密（基于AES-128）存储敏感字段。核心实现位于‘backend/HealthRecordsManager/mcpserver/storage_tool.py’。隐私数据保护核心代码见附录A. 12。HealthDataStorage类实现了加密密钥管理、数据加密和解密三个核心方法。

5.7 本章小结

本章详细阐述了系统的关键实现。A2A服务器和任务管理器实现了多智能体协作框架；MCP客户端和工具封装了外部能力调用；记忆系统支撑了RAG语义检索；前端小程序和SSE流式响应提供了良好的用户体验。通过这些实现，系统具备了健康咨询、档案管理、用药提醒、就诊摘要等完整功能。

7. 第6章系统测试与结果分析

总字符数：1335

相似文献列表

去除本人文献复制比：4%(53)		去除引用文献复制比：4%(53)	文字复制比：4%(53)
1	基于node.js和vue的校园互助微信小程序的设计与实现 吕慧垚 - 《大学生论文联合比对库》 - 2024-05-27		4.0% (53) 是否引证：否
2	裴鑫-201802170145 裴鑫 - 《高职高专院校联合比对库》 - 2021-05-28		3.0% (40) 是否引证：否

原文内容

第6章系统测试与结果分析
6.1 测试环境说明

系统测试是软件开发生命周期中不可或缺的环节，通过测试可以发现系统中的缺陷和错误，保证软件质量。本系统测试主要从功能测试进行验证。

由于测试资源有限，本系统的测试主要在2核8G云服务器环境下进行部署验证。服务器地址为‘http://8.155.166.136:13003’，微信小程序客户端通过该地址访问后端服务。

6.2 部署验证测试

根据系统需求分析阶段确定的功能需求和非功能需求，制定了相应的测试方案和测试用例。测试用例的设计遵循覆盖率高、可操作性强的原则。

6.2.1 服务可用性测试

通过curl命令对部署的服务进行接口验证，结果如表 6- 1 服务可用性测试结果。

表 6- 1 服务可用性测试结果

接口	请求方式	测试结果	响应示例
/ping	GET	正常	"Pong"
/conversation/create	POST	正常	成功创建会话，返回 conversation_id
/message/send	POST	正常	成功发送消息

接口请求方式测试结果响应示例

/ping GET 正常 "Pong"
/conversation/create POST 正常成功创建会话，返回conversation_id
/message/send POST 正常成功发送消息

6.3 功能验证

由于服务器资源限制，以下功能通过本地开发环境验证后，确认可正常部署运行。

6.3.1 健康档案管理

健康档案管理功能验证如表 6- 2 健康档案管理功能验证所示。

表 6- 2 健康档案管理功能验证

功能	验证方式	状态
OCR文字识别	本地测试	可用
结构化数据提取	本地测试	可用
数据加密存储	本地测试	可用

功能验证方式状态

OCR文字识别本地测试可用
结构化数据提取本地测试可用
数据加密存储本地测试可用

6.3.2 智能问诊

智能问诊功能验证如表 6- 3 智能问诊功能验证所示。

表 6- 3 智能问诊功能验证

功能	验证方式	可用
症状咨询	本地测试	可用
知识问答	本地测试	可用
历史记忆	本地测试	可用

功能验证方式可用

症状咨询本地测试可用

知识问答本地测试可用

历史记忆本地测试可用

6.3.3 用药提醒

用药提醒功能验证如表 6- 4 用药提醒功能验证所示。

表 6- 4 用药提醒功能验证

功能	验证方式	状态
提醒创建	本地测试	可用
定时通知	本地测试	可用
服药记录	本地测试	可用

功能验证方式状态

提醒创建本地测试可用

定时通知本地测试可用

服药记录本地测试可用

6.3.4 就诊摘要生成

就诊摘要生成功能验证如表 6- 5 就诊摘要生成功能验证所示。

表 6- 5 就诊摘要生成功能验证

功能	验证方式	状态
病例文档解析	本地测试	可用
摘要自动生成	本地测试	可用

功能验证方式状态

病例文档解析本地测试可用

摘要自动生成本地测试可用

6.4 结果分析与讨论

6.4.1 功能完整性分析

通过对系统各功能模块的测试，验证了系统的功能完整性和正确性。测试结果表明，系统实现了开题报告要求的各项功能指标，能够满足用户的基本需求。

系统实现了开题报告要求的四大核心功能模块：健康档案管理、智能健康咨询、用药提醒、就诊摘要生成。

6.4.2 局限性讨论

尽管系统基本实现了预期功能，但在实际测试过程中也发现了一些局限性和不足之处，需要在后续工作中持续优化和改进。

- （1）OCR识别准确率受图片质量影响较大，低分辨率或倾斜角度的图片识别效果不佳；
- （2）系统目前仅支持中文语言，对于多语言场景支持不足；
- （3）健康咨询功能定位为辅助参考，不能替代专业医疗诊断；

6.5 本章小结

本章展示了系统的测试方案和结果分析。功能测试验证了各模块的正确性。同时也指出了系统的局限性，为后续优化提供了方向。

8. 第7章总结与展望		总字符数：11574
相似文献列表		
去除本人文献复制比：3. 6%(422) 去除引用文献复制比：3. 6%(422) 文字复制比：3. 6%(422)		
1	2550755540976448610176_长期降水变化和氮添加对青藏高原高寒草原植_邓彤彤 长期降水变化和氮添加对青藏高原高寒草原植 - 《大学生论文联合比对库》- 2025-03-24	1. 2% (136) 是否引证：否
2	短视频对内蒙古旅游形象塑造与传播的影响研究 梁中坤 - 《大学生论文联合比对库》- 2025-04-24	1. 1% (132) 是否引证：否

3	湖湘红色文化与初中合唱教学的融合提升策略研究 陈欢欢 - 《大学生论文联合比对库》 - 2024-06-17	1.1% (125) 是否引证: 否
4	医用探测镜的光学设计 丁萍 - 《大学生论文联合比对库》 - 2025-05-21	1.0% (110) 是否引证: 否
5	师范系的毕业论文好写吗.docx. - 《网络 (https://www.renrendo) 》 - 2025	0.9% (100) 是否引证: 否
6	大学校园墙小程序的设计与实现 黄超怡 - 《大学生论文联合比对库》 - 2025-05-22	0.8% (96) 是否引证: 否
7	论文fn.doc 陶文 - 《大学生论文联合比对库》 - 2025-06-02	0.7% (78) 是否引证: 否
8	毕业论文(设计)致谢6篇-202509094.docx. - 《网络 (https://www.renrendo) 》 - 2025	0.7% (76) 是否引证: 否
9	会记毕业论文选题.docx. - 《网络 (https://www.renrendo) 》 - 2025	0.6% (74) 是否引证: 否
10	基于spring boot的药店进销管理系统的设计与实现 雷建芳 - 《大学生论文联合比对库》 - 2024-05-07	0.6% (71) 是否引证: 否
11	“双减”背景下小学数学作业设计与优化研究 黄千格 - 《大学生论文联合比对库》 - 2025-05-10	0.6% (69) 是否引证: 否
12	年产1800千升柿子果酒工厂设计（重点车间：灌装） 安惠敏 - 《大学生论文联合比对库》 - 2025-06-15	0.5% (60) 是否引证: 否
13	刘俊源_22920212204167_基于物理仿真的心脏支架效果预测 刘俊源 - 《大学生论文联合比对库》 - 2025-05-31	0.4% (52) 是否引证: 否
14	基于物理仿真的心脏支架效果预测 刘俊源 - 《大学生论文联合比对库》 - 2025-05-10	0.4% (52) 是否引证: 否
15	中国红茶出口现状及对策研究 代领 - 《大学生论文联合比对库》 - 2025-05-27	0.4% (45) 是否引证: 否
16	离线强化学习策略评估 王维维(导师: 吴贤毅;李育强) - 《华东师范大学博士学位论文》 - 2024-12-01	0.3% (34) 是否引证: 否
17	面向电信行业的商机管理系统设计与实现 - 道客巴巴 - 《互联网文档资源 (http://www.doc88.com) 》 - 2019	0.3% (33) 是否引证: 否
18	低压氧化锌避雷器设计.pdf - 豆丁网 - 《互联网文档资源 (http://www.docin.com) 》 - 2017	0.3% (32) 是否引证: 否
19	优秀主管获奖感言范文（通用16篇） - 《网络 (https://www.ruiwen.c) 》 - 2025	0.3% (30) 是否引证: 否
20	基于交互艺术的粒子动态与视觉表现研究——以毕业设计《浮光灵影》为例 杜鑫怡 - 《大学生论文联合比对库》 - 2025-06-19	0.2% (28) 是否引证: 否
21	2025年智能医疗AI语音交互健康咨询系统开发可行性研究报告.docx. - 《网络 (https://www.renrendo) 》 - 2025	0.2% (24) 是否引证: 否

原文内容

第7章总结与展望

7.1 研究工作总结

随着人工智能技术的快速发展，智能体技术在医疗健康领域的应用日益广泛。本文围绕基于A2A-MCP架构的个人智能健康助手设计与实现这一课题，开展了从需求分析、架构设计、系统实现到测试验证的完整研究工作。

7.1.1 主要完成工作

本文完成了系统的需求分析，明确了健康档案管理、智能健康咨询、用药提醒、就诊摘要生成四大核心功能需求，分析了非功能需求包括性能、安全和可用性要求。完成了系统总体设计，设计了基于A2A-MCP双核的分布式智能体架构，把系统分成接入层、智能体层、工具层和数据层，完成了智能体职责划分和交互协议设计。完成了系统的详细实现，实现了健康顾问、健康档案管理员、用药提醒助手、就诊摘要生成器四个核心智能体，以及OCR、提醒、文档解析等MCP工具调用能力。完成了系统测试与评估，通过功能测试验证了系统的正确性。

7.1.2 主要创新点

本文的主要创新点包括架构创新和工程创新两个方面。在架构创新方面，提出并实现了A2A-MCP融合架构，通过A2A协议解决多智能体协作问题，通过MCP协议解决工具标准化接入问题，实现了职责清晰、易于扩展的系统结构。在工程创新方面，在统一任务链路中融合了流式响应、工具回环和持久化机制，提供了完整的从请求接入到结果返回的端到端解决方案。

7.1.3 实际应用价值

系统已在以下方面展现出应用价值。帮用户便捷管理电子健康档案，减少人工整理负担。通过智能问诊功能提供初步健康建议，提升健康意识。通过用药提醒功能提高服药依从性，改善健康管理效果。为就医提供就诊前摘要准备，提升医患沟通效率。

7.2 存在的不足

尽管系统基本实现了设计目标，但仍存在一些不足。功能完善度有待提升，部分高级功能如健康趋势分析、个性化健康报告等尚未实现，限制了系统的深度应用价值。OCR识别准确率受限，在低质量图片、复杂版式、手写体等场景下，识别准确率仍有提升空间。多语言支持不足，目前仅支持中文，在多语言场景下受限。缺少临床验证，健康咨询功能尚未经过严格的临床验证，实际应用需谨慎。

7.3 未来研究展望

针对上述不足，未来研究可在以下方向开展。增强智能化能力，引入更先进的医学知识图谱和诊断模型，提升健康咨询的专业性和准确性。优化OCR识别，结合版面分析和后纠正模型，提升复杂文档的识别效果。扩展应用场景，探索在慢性病管理、家庭健康管理、医疗资源调度等领域的应用。推进临床验证，与医疗机构合作，对系统进行临床验证和效果评估。完善生态系统，丰富MCP工具生态，接入更多医疗健康服务能力。

7.4 本章小结

本章总结了本文的研究工作和主要成果，指出了系统存在的不足，并对未来研究方向进行了展望。总体而言，本文设计实现的基于A2A-MCP的个人智能健康助手系统，在多智能体协同与工具调用方面做出了有益探索，为相关领域的研究和应用提供了参考。

参考文献

- [1] 周丹, 等. 数字健康技术在主动健康领域的应用进展与发展建议[J]. 中国数字医学, 2023, 18(5): 1-8.
- [2] 李文杰, 等. 人工智能技术在医疗辅助诊断领域的应用现状与趋势分析[J]. 中国医学装备, 2022, 19(12): 25-30.
- [3] 夏鹏, 等. 大语言模型驱动的多智能体协同代码生成技术[J]. 软件学报, 2024, 35(3): 1-18.
- [4] 陈星宇, 等. 融合LLMs和MCP技术的数据分析与知识服务智能体研究与应用[J]. 计算机应用研究, 2024, 41(2): 385-392.
- [5] 满坚平, 等. 智能体在医疗健康领域的研究与应用[J]. 人工智能, 2023, 4(2): 78-89.
- [6] 张泉, 等. 针对大语言模型MCP服务器中权限使用的实证研究[J]. 中文信息学报, 2024, 38(1): 45-56.
- [7] 张雷, 等. 医学人工智能研究的伦理挑战与审查要点[J]. 医学与哲学, 2023, 44(15): 32-37.
- [8] 唐财兴, 等. 大语言模型与模型上下文协议如何重塑医学教育的未来[J]. 医学教育探索, 2024, 23(2): 12-20.
- [9] 陈效良, 等. 基于MCP服务器的大模型与物联网平台交互架构设计与实现[J]. 计算机工程, 2024, 50(4): 112-120.
- [10] 王鑫林, 等. 检索增强生成(RAG)综述: 方法与应用[J]. 计算机学报, 2024, 47(5): 1023-1048.
- [11] 吴国栋, 等. 检索增强生成推荐及其研究进展[J]. 计算机科学与探索, 2024, 18(6): 1325-1340.
- [12] 李鸿亮, 等. 大模型时代的光学文字识别: 现状及展望[J]. 模式识别与人工智能, 2023, 36(8): 701-715.
- [13] 蔡千卓, 等. 基于深度学习的医疗图像文档信息提取关键技术研究[J]. 中国生物医学工程学报, 2023, 42(4): 456-465.
- [14] 许莹, 等. 基于CRAFT和OCR技术的药品名称识别方法[J]. 计算机应用与软件, 2024, 41(3): 178-184.
- [15] 宋子文, 等. 向量数据库中近似最近邻搜索关键技术综述[J]. 软件学报, 2023, 34(9): 2090-2108.
- [16] 唐新宇, 等. 基于协同双源检索的医学教育问答检索增强生成[J]. 现代教育技术, 2024, 34(5): 78-86.
- [17] 陈进, 等. 基于知识图谱的医疗诊断智能问答系统[J]. 计算机应用, 2023, 43(S1): 187-192.
- [18] 叶姗, 等. “智慧服药”提醒系统联合微信群督导模式在颅内动脉支架置入术后患者服药管理中的应用[J]. 护理研究, 2023, 37(20): 3725-3730.

附录

A.1 A2A服务器核心代码

A2A服务器核心类位于‘backend/A2AServer/src/A2AServer/agent.py’，负责智能体初始化、工具连接和任务处理。

A2AServer核心类

```
class BasicAgent:
def __init__(self, config_path, model_name, prompt_file, provider):
self.servers = {} # MCP服务器实例
self.all_functions = [] # 可用工具列表
self.session_conversations = defaultdict(list) # 会话历史
async def setup_tools(self):
# 初始化MCP工具连接
pass
async def handle_task(self, task):
# 处理A2A任务
pass
```

A.2 任务管理器核心代码

任务管理器位于 ‘backend/A2AServer/src/A2AServer/task_manager.py’，继承自 ‘InMemoryTaskManager’ 类，负责跟踪任务状态、管理任务队列。

```
class AgentTaskManager(InMemoryTaskManager):
    """Task manager for AG2 MCP agent."""
    def __init__(self, agent: BasicAgent):
        super().__init__()
        self.agent = agent
    async def on_send_task(self, request: SendTaskRequest) -> SendTaskResponse:
        """处理同步任务请求"""
        validation_error = self._validate_request(request)
        if validation_error:
            return SendTaskResponse(id=request.id, error=validation_error.error)
        await self.upsert_task(request.params)
        await self.update_store(
            request.params.id, TaskStatus(state=TaskState.WORKING), None
        )
        task_send_params: TaskSendParams = request.params
        query = self._get_user_query(task_send_params)
        user_parts = self._extract_user_parts(task_send_params)
        user_id = task_send_params.metadata.get("user_id") if task_send_params.metadata else None
        try:
            # 使用 Agent 的非流式推理以实现同步调用
            final_text = await self.agent.run_inference(
                user_query=query,
                sessionId=task_send_params.sessionId,
                stream=False,
                user_id=user_id,
                user_parts=user_parts,
            )
```

A.3 MCP客户端核心代码

MCP客户端位于 ‘backend/A2AServer/src/A2AServer/mcp_client/client.py’，负责与MCP服务器建立连接、调用工具并处理响应。

```
class SSEMCPCClient:
    """Implementation for a SSE-based MCP server."""
    def __init__(self, server_name: str, url: str):
        self.server_name = server_name
        self.url = url
        self.tools = []
        self.session = None
    async def start(self):
        """建立SSE连接"""
        self._streams_context = sse_client(url=self.url, sse_read_timeout=None)
        streams = await self._streams_context.__aenter__()
        self._session_context = ClientSession(*streams)
        self.session = await self._session_context.__aenter__()
        await self.session.initialize()
        return True
    async def list_tools(self):
        """列出可用工具"""
        if not self.session:
            return []
        return await self.session.list_tools()
```

A.4 OCR识别核心代码

OCR识别工具位于 ‘backend/HealthRecordsManager/mcpserver/ocr_tool.py’，调用阿里云OCR服务进行文字识别。

```
def call_aliyun_ocr(image_base64: str) -> str:
    """调用阿里云OCR服务"""
    access_key_id = os.getenv('ALIYUN_ACCESS_KEY_ID')
    access_key_secret = os.getenv('ALIYUN_ACCESS_KEY_SECRET')
```



```

endpoint = os.getenv('ALIYUN_OCR_ENDPOINT', 'ocr-api.cn-shanghai.aliyuncs.com')
if not access_key_id or not access_key_secret:
    return "阿里云OCR配置缺失"
try:
    from alibabacloud_ocr20191230.client import Client as OCRClient
    from alibabacloud_ocr20191230 import models as ocr_models
    config = ocr_models.Config()
    config.access_key_id = access_key_id
    config.access_key_secret = access_key_secret
    config.endpoint = endpoint
    client = OCRClient(config)
    request = ocr_models.RecognizeAdvancedRequest()
    request.image = base64.b64decode(image_base64)
    response = client.recognize_advanced(request)
    return response.data or "识别结果为空"
except Exception as e:
    return f"阿里云OCR调用失败: {str(e)}"

```

A.5 用药提醒核心代码

用药提醒工具位于 'backend/MedicationReminder/mcpserver/reminder_tool.py'，管理用药计划和发送提醒。

```

# PostgreSQL 连接配置
PG_DSN = os.environ.get("DATABASE_URL") or \
    "postgresql://pha:pha_pass@postgres:5432/personal_health_assistant"
@mcp.tool()
async def create_medication_reminder(...) -> Dict[str, Any]:
    """创建用药提醒"""
    with get_pg_conn() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            # 插入提醒记录
            cur.execute("""
                INSERT INTO reminders
                (user_id, reminder_type, title, reminder_time, repeat_type)
                VALUES (%s, 'medication', %s, %s, %s)
                RETURNING id
            """, (user_id, title, reminder_time, repeat_type))
            reminder_id = cur.fetchone()["id"]
            return {"success": True, "reminder_id": reminder_id}

```

A.6 智能体协作核心代码

智能体协作工具位于 'backend/HealthAdvisor/mcpserver/a2a_integration_tool.py'，通过A2A协议进行任务委托和结果交换。

```

def _send_task(agent_endpoint_url: str, tool_instruction_text: str,
    session_id: Optional[str] = None, user_id: Optional[str] = None) -> Dict[str, Any]:
    """向其他智能体发送任务请求"""
    body = {
        "jsonrpc": "2.0",
        "method": "tasks/send",
        "params": {
            "id": str(uuid.uuid4()),
            "sessionId": session_id or str(uuid.uuid4()),
            "message": {
                "role": "user",
                "parts": [{"type": "text", "text": tool_instruction_text}]
            },
            "metadata": {"user_id": user_id}
        }
    }
    headers = {"Content-Type": "application/json"}
    response = requests.post(
        f"{agent_endpoint_url}/unary",
        json=body,

```

```

headers=headers,
timeout=30
)
return response.json()
@mcp.tool()
async def a2a_send_task(agent_name: str, task_content: str, ...) -> Dict[str, Any]:
    """A2A任务发送工具 - 供健康顾问调用其他智能体"""
    agent_address = _resolve_agent_address(agent_name)
    result = _send_task(agent_address, task_content, session_id, user_id)
    return result

```

A.7 记忆系统核心代码

记忆系统位于 ‘backend/AgentMemorySystem/memory_manager.py’，实现语义记忆存储和检索。

```

class MemoryManager:
    """记忆管理模块 - 负责记忆的维护、优化和分析"""
    def __init__(self, db_config: MemoryDatabaseConfig = None):
        self.db_config = db_config or MemoryDatabaseConfig()
        self.embedding_service = EmbeddingService()
        async def add_memory(self, user_id: str, content: str,
            memory_type: str = "conversation") -> str:
            """添加新记忆"""
            # 1. 生成文本向量
            vector = await self.embedding_service.get_embedding(content)
            # 2. 存储到向量数据库
            memory_id = await self.db_config.insert_memory(
                user_id=user_id,
                content=content,
                vector=vector,
                memory_type=memory_type
            )
            return memory_id
        async def retrieve_similar_memories(
            self, user_id: str, query: str, top_k: int = 5
        ) -> List[Dict[str, Any]]:
            """检索相似记忆"""
            query_vector = await self.embedding_service.get_embedding(query)
            memories = await self.db_config.search_by_vector(
                user_id=user_id,
                query_vector=query_vector,
                top_k=top_k
            )
            return memories

```

A.8 SSE流式响应核心代码

SSE流式响应位于 ‘backend/A2AServer/src/A2AServer/agent.py’，实现服务端流式推送。

```

async def _stream_response_generator(self, sessionId):
    """SSE流式响应生成器"""
    while True:
        generator = await generate_text(
            self.session_conversations[sessionId],
            self.chosen_model,
            self.all_functions,
            stream=True
        )
        accumulated_text = ""
        async for chunk in generator:
            if chunk.get("is_chunk", False):
            if chunk.get("token", False):
            if chunk.get("is_reasoning"):
                yield {"text": chunk["assistant_text"], "type": "reasoning"}
            else:

```

```

yield {"text": chunk["assistant_text"], "type": "normal"}
accumulated_text += chunk["assistant_text"]
tool_calls = chunk.get("tool_calls", [])
if tool_calls:
    for tc in tool_calls:
        tc["type"] = "function"
yield {"text": f"{json.dumps(tool_calls, ensure_ascii=False)}", "type": "tool_call"}
for tc in tool_calls:
    if tc.get("function", {}).get("name"):
        result = await process_tool_call(tc, self.servers, self.quiet_mode)
        if result:
            yield {"text": f"{json.dumps(new_res)}", "type": "tool_result"}

```

A.9 Host Agent API核心代码

Host Agent API位于‘frontend/hostAgentAPI/api.py’，提供智能体管理的RESTful接口。

```

app = FastAPI()
@app.on_event("startup")
async def startup_event():
    """应用启动时初始化"""
    try:
        if health_api and hasattr(health_api, "_warmup_optional_tools"):
            await health_api._warmup_optional_tools()
    except Exception as e:
        logger.warning(f"Startup warmup failed: {e}")
    class ConversationServer:
        def __init__(self, router: APIRouter):
            api_key = os.environ.get("DEEPSEEK_API_KEY", "")
            self.manager = ADKHostManager(api_key=api_key)
            async def _create_conversation(self, request: Request):
                """创建新会话"""
                body = await request.json()
                conversation = await self.manager.create_conversation(body)
                return conversation

```

A.10 医疗文档结构化提取核心代码

医疗文档结构化提取位于‘backend/HealthRecordsManager/mcpserver/data_extraction_tool.py’，从OCR文本中提取结构化医疗信息。

```

@mcp.tool()
def extract_medical_info(text: str) -> str:
    """从OCR文本中提取结构化医疗信息"""
    extracted_info = {
        'document_type': detect_document_type(text),
        'patient_info': extract_patient_info(text),
        'diagnosis': extract_diagnosis(text),
        'medications': extract_medications(text),
        'test_results': extract_test_results(text),
        'doctor_info': extract_doctor_info(text),
        'date': extract_date(text),
    }
    return json.dumps(extracted_info, ensure_ascii=False, indent=2)
def detect_document_type(text: str) -> str:
    """检测文档类型（处方/检查报告/病历等）"""
    if any(kw in text for kw in ['处方', '药品', '用法', '用量']):
        return 'prescription'
    elif any(kw in text for kw in ['检查报告', '化验单', '检验结果']):
        return 'inspection_report'
    elif any(kw in text for kw in ['病历', '诊断', '主诉', '病史']):
        return 'medical_record'

```

A.11 多智能体状态同步核心代码

多智能体状态同步通过任务管理器实现，位于‘backend/A2AServer/src/A2AServer/task_manager.py’。

```

class AgentTaskManager(InMemoryTaskManager):
    """任务状态管理"""
    async def on_send_task(self, request: SendTaskRequest) -> SendTaskResponse:
        await self.upsert_task(request.params)
        await self.update_store(
            request.params.id,
            TaskStatus(state=TaskState.WORKING),
            None
        )
        # 处理任务并更新状态
        final_text = await self.agent.run_inference(...)
        await self.update_store(
            request.params.id,
            TaskStatus(state=TaskState.COMPLETED),
            Artifact(parts=[TextPart(text=final_text)])
        )
        return SendTaskResponse(id=request.id, result=...)

```

A.12 隐私数据保护核心代码

隐私数据保护位于 ‘backend/HealthRecordsManager/mcpserver/storage_tool.py’，采用Fernet对称加密存储敏感字段。

```

from cryptography.fernet import Fernet
class HealthDataStorage:
    def __init__(self):
        self.encryption_key = self.get_or_create_encryption_key()
        self.cipher = Fernet(self.encryption_key)
    def get_or_create_encryption_key(self):
        """获取或创建加密密钥"""
        key_file = os.path.join(os.path.dirname(__file__), "..", "encryption.key")
        if os.path.exists(key_file):
            with open(key_file, 'rb') as f:
                return f.read()
        else:
            key = Fernet.generate_key()
            with open(key_file, 'wb') as f:
                f.write(key)
            return key
    def encrypt_data(self, data):
        """加密数据"""
        if isinstance(data, dict):
            data = json.dumps(data, ensure_ascii=False)
        return self.cipher.encrypt(data.encode('utf-8'))
    def decrypt_data(self, encrypted_data):
        """解密数据"""
        return self.cipher.decrypt(encrypted_data).decode('utf-8')

```

致谢

时光荏苒，四年的大学生活即将画上句号。回首这段求学时光，感慨万千。在这里，我要向所有给予我帮助和支持的人表达最诚挚的感谢。

首先，我要衷心感谢我的指导老师。从选题的确定、研究方案的制定，到系统的设计实现和论文的撰写修改，老师都给予了悉心指导和耐心解答。老师严谨的治学态度、渊博的专业知识和精益求精的工作作风，让我受益匪浅，也将成为我今后学习和工作的榜样。

感谢我的同学和朋友们。在系统开发和论文写作过程中，我们互相讨论、互相鼓励，共同克服了许多困难。那些一起熬夜调试代码、一起讨论技术方案的日子，将成为我珍贵的回忆。

感谢我的家人。是你们一直以来的理解、包容和支持，让我能够安心完成学业。无论遇到什么困难，家永远是我坚强的后盾。

谨向所有关心和帮助过我的人，致以最诚挚的谢意！

2. 去除引用文献复制比:去除系统识别为引用的文献后, 计算出来的重合字符数在总字符数中所占的比例
3. 去除本人文献复制比:去除系统识别为作者本人其他文献后, 计算出来的重合字符数在总字符数中所占的比例
4. 单篇最大文字复制比:被检测文献与所有相似文献比对后, 重合字符数占总字符数比例最大的那一篇文献的文字复制比
5. 复制比按照“四舍五入”规则, 保留1位小数;若您的文献经查重检测, 复制比结果为0, 表示未发现重复内容, 或可能存在的个别重复内容较少不足以作为判断依据
6. 红色文字表示文字复制部分;绿色文字表示引用部分(包括系统自动识别为引用的部分);棕灰色文字表示系统依据作者姓名识别的本人其他文献部分
7. 系统依据您选择的检测类型(或检测方式)、比对截止日期(或发表日期)等生成本报告
8. 知网个人查重唯一官方网站:<https://cx.cnki.net>

知网个人查重服务
官方网址 cx.cnki.net